

Formal Specification of a Radix Tree Limit Order Book

Radix Matching Contracts

July 12, 2026

Abstract

We formalize a radix tree limit order book implementation. The protocol stores bids and asks as binary radix trees over a fixed 64-bit price-time key. Each node is one 32-byte word containing a signed 32-bit logarithmic tick, a 160-bit quantity, a 32-bit same-tick rounding-correction code, and a 32-bit nonce or path suffix. Tick t represents the quote/base price $2^{96t/2^{31}}$. The complete tick domain spans $[2^{-96}, 2^{96})$, with 2^{32} ticks and adjacent ticks separated by $2^{96/2^{31}}$. We specify the state transition system, integer quote calculation, rounding corrections, accounting invariants, nonce rollover, and computational bounds.

1 Formal Overview

Consider a pair of ERC-20 tokens (X, Y) . The token addresses are ordered by address, and the lower address is denoted X , or `token0`. The higher address is denoted Y , or `token1`. A bid buys X and pays Y . An ask sells X and receives Y .

All quantities and prices in this paper are denominated in raw ERC-20 units. If X has d_X decimals and Y has d_Y decimals, then the displayed human price in units of Y per unit of X is

$$P_{\text{human}}(t) = P(t)10^{d_X - d_Y}.$$

The contracts do not query token decimals and do not rescale token transfers.

The protocol state is divided into books. A book is identified by

$$\text{bookId} = H(X, Y, e),$$

where H is `keccak256` and e is the pool epoch. Each book contains two conceptual radix trees, one for bids and one for asks. The two trees share one mapping from node words to branch children.

The engine exposes three state-changing classes of operations:

- (i) fill one incoming order against one routed book;
- (ii) fill a sequence of routed orders and settle each touched token once at the end of the route;
- (iii) cancel a live order or claim the proceeds of a filled order.

Fills may leave unmatched quantity. If resting is permitted, the unmatched quantity is inserted into the active book for the pair. If the routed book has nonce space, the routed book is restable. Otherwise the active pool epoch determines the restable book. When a book reaches nonce 1, it remains matchable and cancelable, but new resting orders move to the next epoch.

Definition 1 (Identifier encoding). Let $\text{word}(a)$ denote the 32-byte EVM word containing address a left-padded to 32 bytes, and let $\text{word}(e)$ denote the 32-byte unsigned integer encoding of epoch e . The pool and book identifiers are

$$\text{poolId} = H(\text{word}(X) \parallel \text{word}(Y)),$$

and

$$\text{bookId} = H(\text{word}(X) \parallel \text{word}(Y) \parallel \text{word}(e)).$$

The initial epoch is 0. Epochs increase by one when the active book has assigned its last restable nonce.

Remark 1 (Specification boundary). This paper specifies the mathematical state machine and arithmetic contracts. It does not reproduce every hexadecimal constant used by the tick conversion library. The byte-exact implementation of $\hat{P}(t)$ is the Solidity library `TickMath32`; generated vectors and an independent high-precision decimal oracle are the conformance tests for that implementation.

2 Node Encoding

Let

$$T = \{-2^{31}, \dots, 2^{31} - 1\}, \quad Q = \{0, \dots, 2^{160} - 1\}, \quad N = \{0, \dots, 2^{32} - 1\}.$$

Definition 2 (Packed node). For $t \in T$, $q \in Q$, $c \in N$, and $n \in N$, define

$$W(t, q, c, n) = (u_{32}(t) \ll 224) + (q \ll 64) + (c \ll 32) + n,$$

where $u_{32}(t)$ is the two's-complement 32-bit representation of t . The field projections are

$$\text{tick}(W) = t, \quad \text{qty}(W) = q, \quad \text{corr}(W) = c, \quad \text{nonce}(W) = n.$$

Definition 3 (Valid incoming order). An incoming order is a word $W(t, q, 0, 0)$ with $q > 0$. The caller chooses t and q . The correction field and nonce field must be zero.

Definition 4 (Resting order). A resting order is a word $W(t, q, 0, n)$ with $q > 0$ and

$$2 \leq n \leq 2^{32} - 1.$$

The contract assigns n from the book's decrementing nonce.

Definition 5 (Branch node). A nonzero word $W(t, q, c, n)$ is a branch node when it has a nonzero left child in the book's tree mapping. The zero word is reserved as a root anchor and is not a branch even when one of its root pointers is nonzero. For an exact branch, the quantity field q is the aggregate base quantity represented by the subtree. The pair (t, n) is the raw address path suffix of the branch. The correction field is nonzero only for same-tick aggregate branches.

The packed word has the following field layout:

bits 224...255	bits 64...223	bits 32...63	bits 0...31
signed tick	quantity	correction code	nonce/path

The word size is fixed at 256 bits. The radix key is fixed at 64 bits: 32 bits of sortable tick and 32 bits of nonce.

3 Tick Price Model

3.1 Real Arithmetic Model

Definition 6 (Mathematical tick price). *For $t \in T$, define*

$$P(t) = 2^{96t/2^{31}}.$$

The minimum tick is

$$P(-2^{31}) = 2^{-96}.$$

The maximum tick is

$$P(2^{31} - 1) = 2^{96-96/2^{31}} < 2^{96}.$$

The adjacent-tick ratio is constant:

$$\frac{P(t+1)}{P(t)} = 2^{96/2^{31}}.$$

Thus the relative tick spacing is

$$2^{96/2^{31}} - 1 \approx 3.09861 \cdot 10^{-8},$$

or approximately 0.000309861 basis points.

Theorem 1 (Quote range). *For every $t \in T$ and $q \in Q$,*

$$0 \leq qP(t) < 2^{256}.$$

Proof. By definition, $q < 2^{160}$ and $P(t) < 2^{96}$. Therefore

$$qP(t) < 2^{160}2^{96} = 2^{256}.$$

□

3.2 Integer Price Code

The implementation does not evaluate $P(t)$ with real arithmetic. Instead, it computes a normalized integer factor $F(t)$ and a shift $S(t)$ such that

$$\hat{P}(t) = \frac{F(t)}{2^{S(t)}}.$$

The implemented bounds are

$$2^{127} \leq F(t) < 2^{129}, \quad 32 \leq S(t) \leq 224.$$

For $t = 0$, $F(t) = 2^{128}$, $S(t) = 128$, and $\hat{P}(0) = 1$.

The tick conversion decomposes $96t/2^{31}$ into an integer exponent and a fractional exponent. Constants encode negative fractional powers in Q128. The implementation composes the fractional factor by nibbles and then chooses a shift so that the factor remains normalized.

The implementation-level conformance condition is:

$$\hat{P}(t) = F(t)2^{-S(t)}$$

with $F(t)$ and $S(t)$ equal to the values returned by `TickMath32.getPriceFactorAtTick(t)`. The generated test vectors allow at most 64 units of error in the Q128 factor relative to the independent decimal oracle. Settlement uses the integer $\hat{P}$ exactly; it does not use the real-valued P at runtime.

Remark 2 (Reference oracle). *The repository includes an independent differential oracle for this conversion. The oracle uses Python decimal arithmetic with precision 180 and computes $2^{96t/2^{31}}$ directly from $\ln 2$. It also checks that generated Solidity vectors, boundary ticks, sampled exponent boundaries, random ticks, monotonicity, factor normalization, and shift bounds agree with the independent calculation within the configured Q128 unit bound. This is an implementation check, not a replacement for the mathematical definition above.*

4 Integer Quote Functions

Let $F = F(t)$ and $S = S(t)$. For an integer quantity q , define

$$M(t, q) = \frac{q^F}{2^S}.$$

Definition 7 (Rounded quote). *The bid-side rounded quote is*

$$Q^+(t, q) = \lceil M(t, q) \rceil.$$

The ask-side rounded quote is

$$Q^-(t, q) = \lfloor M(t, q) \rfloor.$$

The implementation computes these values by a 512-bit product

$$qF = H2^{256} + L$$

and a right shift by S . The value Q^+ adds one iff the discarded low bits are nonzero. The value Q^- does not add this unit.

Definition 8 (Same-tick difference). *For $q_1 \geq q_2$, define*

$$D^+(t, q_1, q_2) = Q^+(t, q_1) - Q^+(t, q_2),$$

and

$$D^-(t, q_1, q_2) = Q^-(t, q_1) - Q^-(t, q_2).$$

These difference functions are used for partial fills and cancel payouts. They make repeated partial fills telescope exactly.

Theorem 2 (Telescoping). *Let*

$$q_0 \geq q_1 \geq \dots \geq q_m.$$

For either rounding direction $r \in \{+, -\}$,

$$\sum_{i=1}^m D^r(t, q_{i-1}, q_i) = Q^r(t, q_0) - Q^r(t, q_m).$$

Proof. The sum is

$$\sum_{i=1}^m (Q^r(t, q_{i-1}) - Q^r(t, q_i)),$$

which cancels adjacent terms. □

5 Same-Tick Correction Code

If a branch contains only one tick, the protocol can represent the exact sum of leaf-level rounded quotes without storing a second aggregate word. The 32-bit correction field stores the difference between rounding the aggregate quantity once and summing the already rounded children.

Definition 9 (Uniform branch). *A branch X is uniform if every leaf below it has the same tick t . Leaves are uniform. A branch with nonzero correction code is uniform.*

Definition 10 (Uniform quote). *For a uniform subtree X , let $L(X)$ be the multiset of live leaves below X , and let*

$$q_X = \sum_{Y \in L(X)} \text{qty}(Y).$$

For a bid subtree, define

$$U^+(X) = \sum_{Y \in L(X)} Q^+(\text{tick}(X), \text{qty}(Y)).$$

For an ask subtree, define

$$U^-(X) = \sum_{Y \in L(X)} Q^-(\text{tick}(X), \text{qty}(Y)).$$

Definition 11 (Correction). *For a bid uniform branch, the stored correction is*

$$C^+(X) = U^+(X) - Q^+(\text{tick}(X), q_X).$$

For an ask uniform branch, the stored correction is

$$C^-(X) = Q^-(\text{tick}(X), q_X) - U^-(X).$$

The branch stores $\text{corr}(X) = C^r(X) + 1$. A zero correction code means that the node is either a leaf or a mixed-tick branch.

Lemma 1 (Correction is nonnegative). *For a uniform bid branch, $C^+(X) \geq 0$. For a uniform ask branch, $C^-(X) \geq 0$.*

Proof. For real numbers a, b ,

$$\lceil a \rceil + \lceil b \rceil \geq \lceil a + b \rceil,$$

and

$$\lfloor a + b \rfloor \geq \lfloor a \rfloor + \lfloor b \rfloor.$$

The result follows by induction over the leaves in the subtree. □

Lemma 2 (Correction bound). *If a uniform branch has m leaves, then*

$$0 \leq C^r(X) \leq m - 1.$$

Proof. Each time two rounded components are aggregated, the difference between rounding separately and rounding once is at most one integer unit for the ceiling case and at most one integer unit for the floor case. Since the branch has m leaves, at most $m - 1$ binary aggregations contribute integer units to the stored correction. □

Since one book can assign at most $2^{32} - 2$ resting nonces,

$$C^r(X) + 1 \leq 2^{32} - 1.$$

The correction code therefore fits in 32 bits.

6 Radix Keys and Branches

The raw path key of a node is

$$K(W) = (u_{32}(\text{tick}(W)) \oplus 2^{31})2^{32} + \text{nonce}(W).$$

This converts signed ticks into unsigned sortable ticks.

The bid sort key is

$$K_B(W) = K(W).$$

The ask sort key is

$$K_A(W) = (2^{32} - 1 - (u_{32}(\text{tick}(W)) \oplus 2^{31}))2^{32} + \text{nonce}(W).$$

Thus the rightmost bid is the highest tick and earliest nonce, and the rightmost ask is the lowest tick and earliest nonce.

Definition 12 (Branch split). *For two distinct sort keys a, b , define*

$$\text{lcp}(a, b) = \min\{d \in \{0, \dots, 63\} : a_d \neq b_d\},$$

where $d = 0$ is the most significant bit. The child with bit 0 at depth d is the left child. The child with bit 1 at depth d is the right child.

Definition 13 (Self-addressed branch). *Let A, B be nonzero child nodes. Let*

$$q = \text{qty}(A) + \text{qty}(B)$$

and

$$k = \max\{K(A), K(B)\}.$$

If the children are uniform with the same tick, let c be the correction code defined above. Otherwise let $c = 0$. The branch word is

$$\text{branch}(A, B) = W(t_k, q, c, n_k),$$

where t_k and n_k are the tick and low nonce fields encoded by the raw key k .

The child pointers are stored in

$$\text{tree}[\text{branch}(A, B)] = (L, R).$$

Except for the zero root anchor, a node is a branch iff its stored left child is nonzero. A branch word may numerically equal a possible order word. Ownership is not keyed by the branch word alone; it is keyed by `keccak256(bookId, original order word)`. Branch structure is keyed by reachability in the book's tree mapping. Cancel and claim search by the 64-bit side sort key (tick, nonce), not by equality with the original 256-bit order word, because a partial fill changes the live quantity while preserving the order's time key.

7 Book State

Definition 14 (Book). *A book has state*

$$B = (\nu, f, T, O),$$

where $\nu \in N$ is the low 32-bit next nonce, f is a set of book-local flags, T is the branch mapping, and O is the owner mapping shared by all books.

The zero node is reserved as the root anchor:

$$T[0].\text{leftNode} = A_0, \quad T[0].\text{rightNode} = B_0,$$

where A_0 is the ask root and B_0 is the bid root.

The low 32 bits of the packed book word are ν . A book with $\nu = 0$ is uninitialized. A book with $\nu = 1$ is exhausted for new resting orders. It remains valid for matching and cancellation.

The flags f include bid and ask right-spine dirty flags and optional hook activation flags. If a side's right-spine dirty flag is clear, every reachable branch on that side is exact. If the flag is set, reachable branch words on the global right spine may have stale quantity, path, or correction fields, but their child pointers remain authoritative. Off-spine branches remain exact. Before same-side insertion, the dirty right spine is materialized into exact aggregate branches.

Definition 15 (Order id). *The owner key for an order word X in book b is*

$$\text{orderId}(b, X) = H(b, X).$$

The owner state is

$$O[\text{orderId}(b, X)] = (\text{owner}, \text{isBid}).$$

8 State Transition System

8.1 Rest

Definition 16 (Rest code). *Let the selected restable book have next nonce ν . If $\nu = 0$, the router initializes that book by setting $\nu = 2^{32} - 1$ and copying the active hook flags. If $\nu \leq 1$ after restable-book resolution, rest reverts. Otherwise, for an incoming order $W(t, q, 0, 0)$, the resting order is*

$$X = W(t, q, 0, \nu).$$

The next nonce becomes $\nu - 1$, the owner entry

$$O[H(\text{bookId}, X)]$$

is set to the maker and side, and X is inserted into the corresponding radix tree.

If the new nonce is 1, the router initializes the next epoch for the pool. The old book is not modified except for hook-flag handling and remains matchable and cancelable.

Resting is valid only if the quote collateral or claim represented by the order is at most `int256.max`. This is a settlement-layer bound: aggregate mathematical quotes fit below 2^{256} , but signed route deltas must fit in `int256`.

8.2 Leaf Match

Definition 17 (Ask leaf fill). *An incoming bid at limit tick t_b crosses an ask leaf Y iff*

$$\text{tick}(Y) \leq t_b.$$

If the incoming remaining quantity is r , the filled base quantity is

$$\delta = \min\{r, \text{qty}(Y)\}.$$

If the leaf is fully consumed, the quote value is

$$Q^-(\text{tick}(Y), \text{qty}(Y)).$$

If the leaf is partially consumed to remaining quantity $q' = \text{qty}(Y) - \delta$, the quote value is

$$D^-(\text{tick}(Y), \text{qty}(Y), q').$$

Definition 18 (Bid leaf fill). *An incoming ask at limit tick t_a crosses a bid leaf Y iff*

$$\text{tick}(Y) \geq t_a.$$

The filled base quantity is

$$\delta = \min\{r, \text{qty}(Y)\}.$$

If the leaf is fully consumed, the quote value is

$$Q^+(\text{tick}(Y), \text{qty}(Y)).$$

If the leaf is partially consumed to remaining quantity q' , the quote value is

$$D^+(\text{tick}(Y), \text{qty}(Y), q').$$

If $\delta < \text{qty}(Y)$, the live leaf is replaced by

$$W(\text{tick}(Y), \text{qty}(Y) - \delta, 0, \text{nonce}(Y)).$$

The owner mapping remains keyed by the original full-quantity order. The replacement is propagated upward. A parent with two surviving children is rewritten as the self-addressed branch for those children. A parent with one surviving child collapses to that child. Obsolete mapping entries may remain in storage, but they are unreachable from the side root and therefore are not part of the live book.

8.3 Subtree Match

Matching descends through the right child first. That child is the best side for both bid and ask books under the side-specific keys.

A same-tick subtree can be consumed as one aggregate if its whole quantity fits and its tick crosses the incoming limit. Its quote value is obtained from the aggregate quantity and the correction code. A mixed-tick off-spine subtree can be structurally removed as one subtree when its whole quantity fits and its worst executable leaf crosses the incoming limit. The worst executable leaf is the leftmost leaf in the side-specific radix ordering. For an ask subtree, the condition is that the highest ask in the subtree is at most the incoming bid limit. For a bid subtree, the condition is that the lowest bid in the subtree is at least the incoming ask limit. The packed tick of a mixed branch is only an address suffix and is not a crossing bound. The exact quote value of a mixed subtree is computed by recursively summing children. Uniform children terminate this quote recursion in constant time through their correction code.

Definition 19 (Linear reference semantics). *For a bid taker, sort ask leaves by increasing tick and then decreasing nonce. For an ask taker, sort bid leaves by decreasing tick and then decreasing nonce. The reference match consumes the longest prefix of crossing leaves, possibly followed by one partial leaf, whose total base quantity does not exceed the incoming quantity.*

Theorem 3 (Price-time priority). *For a well-formed book, the right-first radix traversal consumes leaves in the same order as the linear reference semantics.*

Proof. For bids, K_B orders larger ticks to larger keys and preserves larger nonce as earlier priority at the same tick. For asks, K_A reverses the tick order and preserves the nonce order. A binary radix tree ordered by these keys has the rightmost leaf as the maximum key. Right-first traversal therefore enumerates best price first and earlier nonce first at equal price. $\square$

Corollary 1 (Best executable price). *Let Y_i and Y_j be two resting leaves consumed by one match, with Y_i consumed before Y_j . If the incoming order is a bid, then*

$$\text{tick}(Y_i) \leq \text{tick}(Y_j).$$

If the incoming order is an ask, then

$$\text{tick}(Y_i) \geq \text{tick}(Y_j).$$

At equal tick, $\text{nonce}(Y_i) \geq \text{nonce}(Y_j)$. Thus no consumed leaf is executed after another consumed leaf with a strictly better price for the incoming order, and equal-price leaves are consumed in time-priority order.

8.4 Cancel and Claim

Cancel is also the maker claim operation. The caller supplies the original order word and the book epoch. The contract computes the order id and checks ownership.

Let the original order be $X = W(t, q, 0, n)$. Let r be the live remaining quantity found in the tree. If no live leaf is found, let $r = 0$. The filled quantity is

$$f = q - r.$$

If $r > q$, the operation reverts.

For a bid maker, the payout is

$$\text{base} = f, \quad \text{quote} = Q^+(t, r).$$

For an ask maker, the payout is

$$\text{base} = r, \quad \text{quote} = D^-(t, q, r).$$

After this computation, the order state is deleted. If an external token transfer reverts, the transaction reverts and the deletion is reverted with it.

9 Routing, Fees, and Hooks

9.1 Pool and Book Routing

The router validates that $0 < X < Y$ by address. It does not check token code. Token admissibility is a deployment configuration condition. The core contracts do not enforce an allowlist and do not verify token balance deltas.

For a fill leg, the caller supplies X , Y , an epoch, an order, a side, a no-rest flag, and a fill-or-kill flag. The routed book id is

$$H(X, Y, e).$$

If the routed book is uninitialized, matching cannot occur. Such a leg is valid only if the order is allowed to rest, is not fill-or-kill, and the routed epoch is the active epoch for the pair. In that case the rest path initializes the book before assigning the resting nonce.

For a route of several legs, all book mutations happen before ERC-20 settlement. The route accumulates signed deltas by token in transient storage:

$$\Delta_Z = \sum_i \Delta_{Z,i}.$$

At the end, positive deltas are transferred to the caller and negative deltas are pulled from the caller. Each touched token is settled once per sign pass.

9.2 Fees

The fee configuration is a single packed word containing a recipient and a basis-point value b . The maximum is

$$0 \leq b \leq 100.$$

A zero recipient disables fees.

Fees are charged only on the taker's positive matched output. For a bid taker, the fee token is X . For an ask taker, the fee token is Y . Cancel and claim do not charge fees.

In a route, the fee is computed per fill leg before route settlement deltas are aggregated. Fee rounding is therefore per-leg, not per-route or per-token.

Definition 20 (Fee). *For taker output a , the fee is*

$$\phi(a, b) = \left\lfloor \frac{ab}{10000} \right\rfloor.$$

The implementation computes this value as

$$\left\lfloor \frac{a}{10000} \right\rfloor b + \left\lfloor \frac{(a \bmod 10000)b}{10000} \right\rfloor,$$

which is equal to $\lfloor ab/10000 \rfloor$ and avoids overflowing when a is near the signed settlement bound.

9.3 Hooks

Hooks are optional and pool scoped. A pool can enable hooks for top bid changes, top ask changes, or both. Hook flags are copied into canonical books. Older epochs remain matchable and cancelable but do not acquire new hook activation after rotation.

The core matcher records only

(outgoingAmount, incomingNonce)

in transient storage when the top order changes. The router adds `poolId`, `bookId`, token address, and hook address after the tree mutation returns. Hook calls are wrapped in `try/catch` and are gas capped. A hook failure does not revert the matching transition.

The hook token identifies the top-buyer side that changed: X for top bids and Y for top asks. The outgoing amount is the live base quantity encoded in the displaced top order. It is not a mixed-subtree quote notional. The incoming nonce is the nonce of the new top order, or zero if the side is empty.

The implementation uses transient storage for the reentrancy guard, route deltas, and hook-change handoff. It therefore assumes an EVM with EIP-1153 transient storage semantics.

10 Accounting

Assumption 1 (Exact ERC-20 transfers). *Every configured token debits the sender and credits the receiver by the exact requested amount, or reverts. Fee-on-transfer, rebasing, and malicious tokens are outside this accounting model.*

Assumption 2 (No unsolicited transfers for equality). *No external actor transfers tokens directly to the contract except through successful protocol calls. If unsolicited transfers occur, the balance equality below becomes a lower-bound statement: balances may exceed obligations.*

Let O_B be the set of active bid orders and O_A be the set of active ask orders in all books. For an order i , let q_i be its original quantity and r_i its live remaining quantity. If an order is fully filled and not yet claimed, $r_i = 0$. Let t_i be its tick.

The contract's token X obligations are

$$\sum_{i \in O_A} r_i + \sum_{i \in O_B} (q_i - r_i).$$

The first term is unfilled ask collateral. The second term is filled base owed to bid makers.

The contract's token Y obligations are

$$\sum_{i \in O_B} Q^+(t_i, r_i) + \sum_{i \in O_A} D^-(t_i, q_i, r_i).$$

The first term is unfilled bid collateral. The second term is filled quote owed to ask makers.

Theorem 4 (Accounting stability). *Under exact ERC-20 transfers and no unsolicited transfers, after every successful fill, route, cancel, or claim, the contract token balances equal the outstanding obligations defined above, excluding fees already transferred to the fee recipient. Without the no-unsolicited-transfer assumption, balances are greater than or equal to the outstanding obligations.*

Proof. The proof is by induction over state transitions.

Resting an ask increases contract token X balance by q and adds the obligation $r = q$ to the first X term. Resting a bid increases contract token Y balance by $Q^+(t, q)$ and adds the same amount to the first Y term.

Matching an ask leaf with old remaining quantity q_1 and new remaining quantity q_2 transfers $q_1 - q_2$ units of X to the bid taker and pulls $D^-(t, q_1, q_2)$ units of Y from the bid taker. The ask's

X obligation decreases by $q_1 - q_2$, and its Y claim increases by $D^-(t, q_1, q_2)$. Matching a bid leaf is dual: the bid's Y collateral decreases by $D^+(t, q_1, q_2)$, and its X claim increases by $q_1 - q_2$. By the telescoping theorem, multiple partial fills produce the same total claim as one fill from original quantity to final remaining quantity.

Canceling or claiming deletes the order state and transfers exactly the remaining obligation computed from r . If a transfer reverts, the EVM reverts the deletion and all earlier writes in the transaction. Fees are deducted from taker output after match accounting has been computed and do not change maker obligations. $\square$

11 Computational Complexity

Let

$$h = 64$$

be the radix key width. Let m be the number of mixed-tick leaves or mixed-tick internal components whose quote values must be visited to compute an exact mixed-tick quote sum.

Theorem 5 (Insertion and cancel depth). *Insertion and exact-key removal touch at most h radix depths.*

Proof. At each branch, the algorithm either terminates, splits at the first earlier differing bit, or descends to exactly one child selected by the next key bit. The split depth strictly increases on descent and is bounded by the 64-bit key width. $\square$

Theorem 6 (Uniform aggregate matching). *If every consumed aggregate subtree is uniform, matching work is bounded by $O(h)$ radix decisions plus a constant amount of quote work per aggregate subtree.*

Proof. Uniform quote values are represented by aggregate quantity and a 32-bit correction code. Therefore consuming a uniform subtree does not require visiting each leaf. The traversal follows right-first radix paths whose depth is bounded by h . $\square$

Theorem 7 (Mixed quote computation). *For mixed-tick subtrees, exact quote computation is $O(m)$ in the number of mixed components visited by the recursive quote function.*

Proof. A mixed branch has no single tick and no correction code that can represent the sum of child rounded quote values. The implementation therefore recursively computes the quote value of the right child and the left child. Recursion terminates at leaves and uniform branches. Thus the work is proportional to the number of visited mixed components. $\square$

There is no separate resumable cursor in the core state machine. If a transaction chooses a path whose mixed-component work exceeds the available gas, the transaction reverts and no partial progress is committed. The fixed 64-bit key depth bounds path selection and exact same-tick aggregate operations; it does not by itself bound the number of mixed quote components visited in a fully exact mixed subtree.

11.1 Pathological Cases

The key width bounds tree depth, but it does not make every exact quote calculation independent of subtree composition.

- (i) A same-tick nonce comb can have depth up to 64, but uniform correction codes allow aggregate quote calculation without visiting every leaf.
- (ii) A mixed-tick comb can force traversal along the full key depth and can require recursive quote calculation for mixed components.
- (iii) A dirty right spine can have stale branch words along the best side. Matching a same-tick dirty spine recovers quantity and quote through right-child recursion bounded by depth. Matching a mixed-tick dirty spine falls back to recursive matching because a stale mixed branch has no valid aggregate quote code.
- (iv) Same-side insertion after right-spine optimization materializes the dirty right spine before inserting the new leaf. This work is bounded by the radix depth.

The theoretical maximum number of resting leaves in one book is

$$2^{32} - 2,$$

because nonce 0 is reserved for incoming orders and uninitialized books, and nonce 1 is the exhausted sentinel. This nonce bound also bounds the maximum number of leaf-level rounding units represented by a same-tick correction.

12 Integer Safety

All live quantities are bounded by $2^{160} - 1$. The mathematical price range is below 2^{96} . Therefore aggregate quote values are below 2^{256} . The quote code computes the product with a high and low word before shifting. It does not require the unshifted product to fit in 256 bits.

The routing layer separately requires final signed deltas to fit in `int256`. If a fill would produce a positive or negative token delta outside that range, the route reverts and all book writes in the transaction are reverted.

Branch quantities are computed by checked 160-bit addition. If a branch would exceed $2^{160} - 1$, the state transition reverts. This prevents silent quantity wraparound.

13 Conclusion

The protocol is a fixed-key radix order book with logarithmic key depth and exact integer accounting. The price domain has 2^{32} logarithmic ticks over $[2^{-96}, 2^{96})$. The quantity domain is 160 bits. Each book has $2^{32} - 2$ assignable resting nonces before rollover. The node word allocates the remaining 32 bits to same-tick rounding correction.

The main accounting property follows from using differences of rounded same-tick notionals. Partial fills telescope, cancel uses the original order and live remaining quantity, and aggregate same-tick branches store the correction required to recover the sum of leaf-level rounded quotes. Mixed-tick quote calculation remains exact by recursion. The computational bound for insertion and cancel is the 64-bit radix depth. Matching is bounded by radix depth for uniform aggregate paths and by the number of mixed components whose exact quote must be computed for mixed-tick subtrees.

References

- [1] Yi Zhang, Xiaohong Chen, and Daejun Park. Formal specification of constant product ($x \times y = k$) market maker model and implementation. Runtime Verification, 2018.
- [2] Donald R. Morrison. PATRICIA – Practical algorithm to retrieve information coded in alphanumeric. *Journal of the ACM*, 15(4):514–534, 1968.
- [3] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core. Whitepaper, 2021.